Dossier de préparation

Deuxième phase

Equipe 8

ARAVENA BRAVO Damien
JACOB Timothé
BENDRIS Nael
TIERRIE Axel
BENKHEIRA Lilya
DESCHAUX-BEAUME Maëlys
SEKMA Sarah

Table des matières

Préambule 2

Cadrage du projet 3

Contraintes et risques 4

Rétrospective sur les risques 5

Profilage des utilisateurs 6

Profils ciblés des utilisateurs finaux 7

Analyse des besoins 8

Modélisation des processus 9

Besoins fonctionnels 10

Analyse de l'existant 11

Stratégies de réduction 12

Besoins non-fonctionnels 13

Conception 14

Maquettes 15

Mise en oeuvre des besoins non-fonctionnels 17

Architecture du code 19

Implémentation de la base de données 20

Etude de faisabilité technique 22

Analyse des besoins techniques 23

Evaluation des contraintes techniques 23

Infrastructure existante 26

Tests de l'infrastructure 28

Sécurité et conformité 29

Environnement de travail 30

Analyse de l'effectué 31

Annexes 33

Matrice RACI 34

Diagramme de cas d'utilisations 36

Répartition des rôles 37

Conduite du projet 38

Diagramme BPMN - 1 40

Sitemap 41

Schéma Entités-Associations 42

Maquettes 43

Sources 48

Préambule

Nous sommes une équipe de développeurs sollicitée par un client dont la demande est de **développer une application** afin de **valoriser** le **patrimoine culturel**.

Valoriser le patrimoine culturel, c'est lui donner une **nouvelle vie**, le faire connaître, le faire aimer et le rendre accessible au plus grand nombre. C'est aussi le **préserver** en fournissant les ressources nécessaires à sa préservation et l'intégrer dans le développement économique et social d'un territoire.

Nous avons suggéré d'axer cette demande sur les **Sites Patrimoniaux Remarquables** (SPR) situés en **Isère**. Ces sites incluent des villes comme Grenoble, et des villages comme Saint-Antoine-l'Abbaye, dont la conservation et la valorisation représentent un **intérêt public** sur le plan historique, architectural et culturel.

Suite au premier document, nous avons réajusté quelques points fondamentaux, comme les **besoins non fonctionnels**. Nous avons synthétisé les autres informations et rédigé une rétrospective sur l'analyse des risques. Pour un détail des informations réduites, veuillez vous référer au document de la phase 1.

Ce rapport se concentrera sur les points suivants :

- **Expression et modélisation des besoins**
    - Modélisation des processus
    - Modèles de données
    - Maquettes
    - Plan du site (Treemap)
- **Réalisation**
    - Étude de faisabilité technique
    - Choix justifié des technologies utilisées
    - Architecture logicielle
    - Architecture réseau
    - Planification avec analyse de l'effectué et du restant

Cadrage du projet

Synthèse des objectifs et du cadre de réalisation

Contraintes et risques

Contraintes	**Risques induits**	
Nom de la contrainte	Nom et index du risque	Mitigation
Travail en équipe de 7 personnes	**R1** Manque de coordination	**EVITEMENT** Établir des règles de travail pour la collaboration, comme des réunions hebdomadaires
Travail d'équipe en présentiel	**R2** Absences, maladies... : risque de désorganisation	**ACCEPTATION** Communiquer pour prévoir les éventuelles absences (Discord), se réorganiser rapidement
Date de rendus	**R3** Dépassement des délais	**RÉDUCTION** Élaboration d'un calendrier (Notion) précis en corrélation avec les dates de rendus imposées
Rendu de documents au format PDF et sous la bonne nomenclature	**R4** Non-conformité de la nomenclature	**PROTECTION** Effectuer des vérifications avant chaque rendu, en communiquant avec Discord
Respect des conventions et règles pour les modèles et documents	**R5** Non-respect des conventions	**RECHERCHE** Solliciter des experts, accéder à la documentation UML
Cohérence des modèles	**R6** Incohérence des diagrammes UML	**RÉDUCTION** Sessions régulières d'analyse des diagrammes UML avec les membres de l'équipe et experts
Exigences de rédaction pour les diagrammes UML (anglais), et les rapports (français).	**R7** Non-cohérence linguistique	**PROTECTION** Convenir de la langue choisie pour chaque partie. Cela passe par une bonne communication d'équipe.
Présentation orales	**R8** Difficultés d'élocutions ou de restitution, donc mauvais rendu oral	**RÉDUCTION** Organiser répétitions et séances d'entraînement, travailler sur la répartition des rôles.
Disponibilité des ressources sur la plateforme Chamilo	**R9** Manque de ressources, ressources inaccessibles	**PROTECTION** Assurer un accès aux ressources nécessaires en téléchargeant les fichiers au préalable.
Travail en distanciel	**R10** Risque de défaillance des services utilisés (Notion, Git...)	**PROTECTION** Utilisation temporaire de services moins adaptés
Intégrité des données	**R11** Problème de données corrompues ou perdues	**EVITEMENT** Faire des sauvegardes de nos travaux, utiliser des outils de versionnage (Git)
Technologies nécessaires à une app web dynamique (HTML/CSS/JS - React)	**R12** Faible maîtrise des technologies	**RECHERCHE** Assistance par IA, aide des experts
Se conformer à la loi	**R13** Utilisation de ressources protégées par des droits d'auteurs	**RÉDUCTION** Bien vérifier, sourcer quand il faut
	R14 Ne pas respecter les données des utilisateurs en matière de RGPD	**RÉDUCTION** Bien expliciter ou transitent les données de l'utilisateur, surtout sur le questionnaire et les avis

Rétrospective sur les risques

A l'issue de cette première phase, parmi nos risques explicités ci-dessus, nous procédons à une analyse détaillée des risques qui se sont **matérialisés**, ainsi que d'autres qui ont pu être **maîtrisés** de manière efficace.

C'est à travers cette rétrospective que nous sommes en mesure d'identifier les points faibles et les bonnes pratiques à conserver pour la suite de ce projet.

Coordination et organisation

Le manque de coordination n'a pas été un problème au sein de l'équipe projet grâce à l'utilisation de nombreux outils d'organisation propres aux enseignements agiles, tels que l'utilisation d'un tableau Kanban.

Au sein de l'équipe projet, les risques de désorganisation ont été vraiment minimes grâce au faible taux d'absentéisme de l'équipe. Les dates de rendus fixées ont également été respectées grâce à l'élaboration d'un calendrier reflétant nos échéances.

Assurance qualité

Nous avons effectué des vérifications tant au niveau du contenu qu'au niveau du nommage des fichiers. De ce fait, la charte graphique a été respectée lors de la fusion des différentes parties prenantes de cette première phase.

Communication et présentation

Pour les présentations orales, nous avons pris soin de désigner uniquement les membres de l'équipe projet à l'aise à l'oral. Cela a permis de garantir une meilleure clarté dans la présentation du sujet et d'optimiser au maximum la qualité du rendu final.

Organisation du travail à distance

Pour la partie travail à distance, nous avons veillé à organiser efficacement des séances, notamment pour préparer les présentations orales finales.

Documents externes

Au cours de cette première phase, nous n'avons rencontré aucun problème lié aux documents déposés sur Chamilo par les enseignants, ce qui a permis d'assurer une continuité académique du projet sans interruption négative tout au long du projet.

Profilage des utilisateurs

Identification et analyse des utilisateurs finaux avec des personas

Profils ciblés des utilisateurs finaux

Cette partie présente les profils d'utilisateurs identifiés pour l'application et leurs attentes principales. Des informations plus détaillées sont disponibles dans le premier document.

Ces éléments guident directement les choix de conception, en lien avec les besoins non-fonctionnels définis.

Amateurs de camping-car

Les amateurs de camping-car, souvent des couples de jeunes retraités, recherchent des suggestions de prochaines étapes adaptées à leurs préférences. Ils apprécient les **interfaces simples** et les **outils guidés**, comme des **questionnaires**, pour décider facilement de leurs itinéraires.

Familles

Les familles souhaitent des visites éducatives et adaptées aux enfants. Elles recherchent des **avis fiables**, des **recommandations de guides** capables de captiver petits et grands, et des **suggestions de restaurants** validées par d'autres familles pour garantir une expérience agréable et enrichissante.

Personnes en situation de handicap moteur

Les personnes à mobilité réduite aspirent à explorer des lieux accessibles tout en disposant d'**informations complètes** et fiables sur les dispositifs mis en place. Elles attendent également des contenus culturels précis, dans une interface adaptée à leurs besoins spécifiques.

Guides

Les guides professionnels souhaitent des **outils simples pour gérer** leurs créneaux, leurs réservations et attirer de nouveaux visiteurs. Les guides amateurs, souvent passionnés par leur patrimoine local, cherchent à partager leurs connaissances dans un cadre convivial, avec une **interface simple d'utilisation**.

Lien avec les besoins non-fonctionnels

Ces différents profils soulignent des besoins essentiels : une **simplicité d'utilisation** pour garantir l'accessibilité (amateurs de camping-car, guides amateurs), une **interface ergonomique** respectant les critères de Bastien et Scapin afin de répondre aux attentes variées (familles, guides professionnels), et une **exactitude des informations** pour assurer une expérience fiable et de qualité (personnes en situation de handicap, familles).

Analyse des besoins

Analyse et identification des besoins utilisateurs

Modélisation des processus

Afin d'avoir une idée plus claire de nos besoins fonctionnels, nous avons réalisé une modélisation de **deux** de nos processus impliquant plusieurs acteurs, car ce sont les deux seuls ayant une présence de plusieurs acteurs, qu'ils soient internes ou externes.

Processus 1

Le premier processus représenté est le fonctionnement de l'application pour une réservation de visite.

Une fois sur l'application, l'utilisateur peut consulter la page à propos du SPR de son choix. Sur cette page, il peut choisir de réserver un guide. S'il est connecté à un compte, il accède directement à l'interface de choix, où il renseigne la date à laquelle il souhaite que la visite se réalise et le guide qu'il souhaite réserver.

En revanche, s'il n'est pas connecté, il lui est proposé de se connecter ou de créer un compte. Une fois cela fait, il accède à l'interface mentionnée précédemment. Une fois la réservation de la visite confirmée et payée en ligne, l'utilisateur reçoit un mail récapitulatif avec sa facture.

Pour finir, Dauphinéo ajoute la visite au dashboard du guide concerné.

Processus 2

Le second processus que nous avons représenté est celui où un utilisateur réalise la démarche afin de devenir un guide local, pour le SPR où il habite par exemple.

L'utilisateur commence par se rendre sur la page du SPR pour lequel il souhaite candidater. Puis, après un clic sur le bouton "se proposer en tant que guide", il accède à un questionnaire où il renseigne ses motivations et parle de lui.

Une fois le formulaire rempli, la candidature est transmise à l'association de guides avec laquelle nous travaillons.

Une fois la décision prise, elle est transmise à l'utilisateur par mail, et dans le cas où il est accepté, son compte est transformé en compte utilisateur-guide et il est ajouté à la base de données en tant que guide.

Les deux diagrammes sont disponibles en version plus détaillés en annexe (p. 40)

Besoins fonctionnels **(hiérarchisés par ordre de priorité)**

Notre diagramme des cas d'utilisation est disponible dans les documents annexes. (p.36)

Système de guides

1. **Se proposer en tant que guide local** (UC1) : un utilisateur voulant faire visiter le SPR où il habite peut se proposer de prendre en charge des visites
2. **Réserver un guide local ou officiel** (UC2) : permettre aux utilisateurs de réserver et payer un guide pour chacun des SPR à tout moment.

Fiche d'information détaillée

3. **Visionner fiche d'informations** (UC3) : permettre à l'utilisateur de bénéficier d'une fiche d'informations relatives à la totalité des SPR contenus en Isère.

Système d'avis utilisateur

4. **Consulter les avis des autres utilisateurs** (UC4) : permettre aux utilisateurs de se référer à des avis relatifs aux différents SPR pour bénéficier de conseils, échanger leurs avis, et recommandations.
5. **Poster son avis et donner une note** (UC5) : l'utilisateur peut donner son avis sur le SPR, qui sera ensuite visible par les autres utilisateurs qui pourront éventuellement y répondre.

Filtrage des SPR

6. **Créer option de filtrage** (UC6) : permettre aux utilisateurs de filtrer le type de SPR désiré, le type de paysage, l'accès aux PMR, etc...

Questionnaire

7. **Bénéficier de recommandations personnalisées** (UC7) : permettre à l'utilisateur de remplir un questionnaire personnalisé qui lui permettra de bénéficier de recommandations personnalisées et basées sur ses réponses au questionnaire.

Outils pour les guides

8. **Consulter le dashboard de visite** (UC8) : les guides peuvent consulter les visites qu'ils devront réaliser.
9. **Renseigner ses disponibilités** (UC9) : permettre aux guides de choisir les plages horaires où ils sont disponibles.

Analyse de l’existant

Dans un contexte centralisé sur la **valorisation du patrimoine culturelle**, de nombreuses applications dans le domaine du tourisme ont su répondre aux besoins croissants des utilisateurs en offrant des services variés que nous détaillerons.

Identification des solutions similaires

Voici un tableau identifiant les fonctions proposées chez les solutions similaires déjà existantes, en comparaison aux fonctions couvertes par notre application.

Solutions	Fiches d'infos détaillées	Recommandations personnalisées	Réservation de guides	Poster des avis
Trip Advisor	Oui	Oui	Non	Oui
AirBNB Experiences	Oui	Partiellement	Oui	Oui
Booking	Oui	Partiellement	Non	Oui
Dauphinéo	Oui	Oui	Oui	Oui

Comment se démarque-t-on des autres ?

Notre application **Dauphinéo** se distingue grâce à 4 critères clés, nous proposons pour nos utilisateurs :

- Des fiches détaillées pour explorer chaque site patrimonial.
- Des recommandations personnalisées adaptées aux préférences.
- Une option de réservation de guides pour des visites interactives.
- Un espace d'avis favorisant échanges et fidélité.

Stratégies de réduction

Voici un rappel des stratégies de réduction prévues pour notre application. Une version plus complète est disponible dans le document de la phase 1.

Se proposer en tant que guide local

Réduction : Limiter les guides à ceux certifiés, sans possibilité pour les utilisateurs de se proposer.

Réserver un guide local ou officiel

Réduction : Restreindre la réservation aux guides officiels disponibles dans une base de données.

Visionner fiche d'informations

Réduction : Ne montrer que les informations essentielles (nom, localisation, description courte, image principale).

Consulter les avis des autres utilisateurs

Réduction : Afficher uniquement les trois avis récents sans options de tri ou de filtrage.

Filtrage des SPR

Réduction : Proposer uniquement des filtres basiques comme le type de site et la distance.

Créer option de filtrage

Réduction : Limiter les filtres à quelques catégories principales, sans combinaisons avancées.

Répondre au questionnaire

Réduction : Simplifier à 3-4 questions clés pour des recommandations génériques.

Consulter le dashboard de visite

Réduction : Restreindre aux visites planifiées et sites favoris, sans statistiques ni suggestions avancées.

Renseigner ses disponibilités

Réduction : Proposer une simple sélection de date et heure sans synchronisation ou préférences avancées.

Besoins non-fonctionnels

Les besoins non-fonctionnels de notre application s'appuient principalement sur les attentes et les contraintes identifiées à partir des personas. Ces besoins visent à garantir une expérience utilisateur fluide, un fonctionnement performant et une qualité optimale du produit final.

Accessibilité

L'application doit être pensée pour être accessible. Nous ferons des choix de design adaptés (contrastes élevés, texte lisible) et la compatibilité avec les lecteurs d'écrans.

Ergonomie

Pour garantir une bonne ergonomie, nous choisissons d'appliquer les critères de Bastien et Scapin, tels que :

- **Guidage** : en prêtant attention à l'incitation (icônes et messages explicites) et en apportant un feedback immédiat et clair, notamment en cas d'erreur de saisie.
- **Charge de travail** : il faut a tout pris éviter une surcharge informationnelle, sauf éventuellement lors de la description détaillée des sites : cas exceptionnel dans lequel nous mettrons aussi en place un résumé
- **Cohérence** : l'interface doit à tout prix maintenir une cohérence visuelle et fonctionnelle. Cela doit permettre à l'utilisateur de savoir rapidement que fait quoi, tout en étant en adéquation avec la charte graphique du projet.
- **Contrôle explicite** : les commandes doivent être clairement identifiables, doivent informer l'utilisateur explicitement de l'action, tout en laissant le choix de modifier ou d'annuler l'action.
- **Signifiance des codes et dénominations** : les icônes et labels utilisés doivent être compréhensibles et suivre les conventions et bonnes pratiques.

Exactitude

Les informations affichées doivent être précises et fiables, et nous devons garantir une **gestion des erreurs**, afin d'assurer la fiabilité des informations. L'interface doit garantir que seules les données valides saisies par l'utilisateur soient traitées et que les informations communiquées par l'utilisateur soient conformes.

Performances

L'application doit pouvoir évoluer pour gérer un volume croissant d'utilisateurs et de contenu, sans compromettre les performances ni la stabilité.

Conception

Représentations structurelles et fonctionnelles

Maquettes

Nous allons maintenant présenter les différentes **maquettes** du site, en justifiant les choix faits. Ceux en rapport avec les critères de **Bastien et Scapin** sont plus détaillés dans la section juste après "*Mise en œuvre des besoins non-fonctionnels*". (p.17)
Une sitemap est disponible en annexe (p.41).

Page d'accueil

Nous avons opté pour ce design simple et élégant.

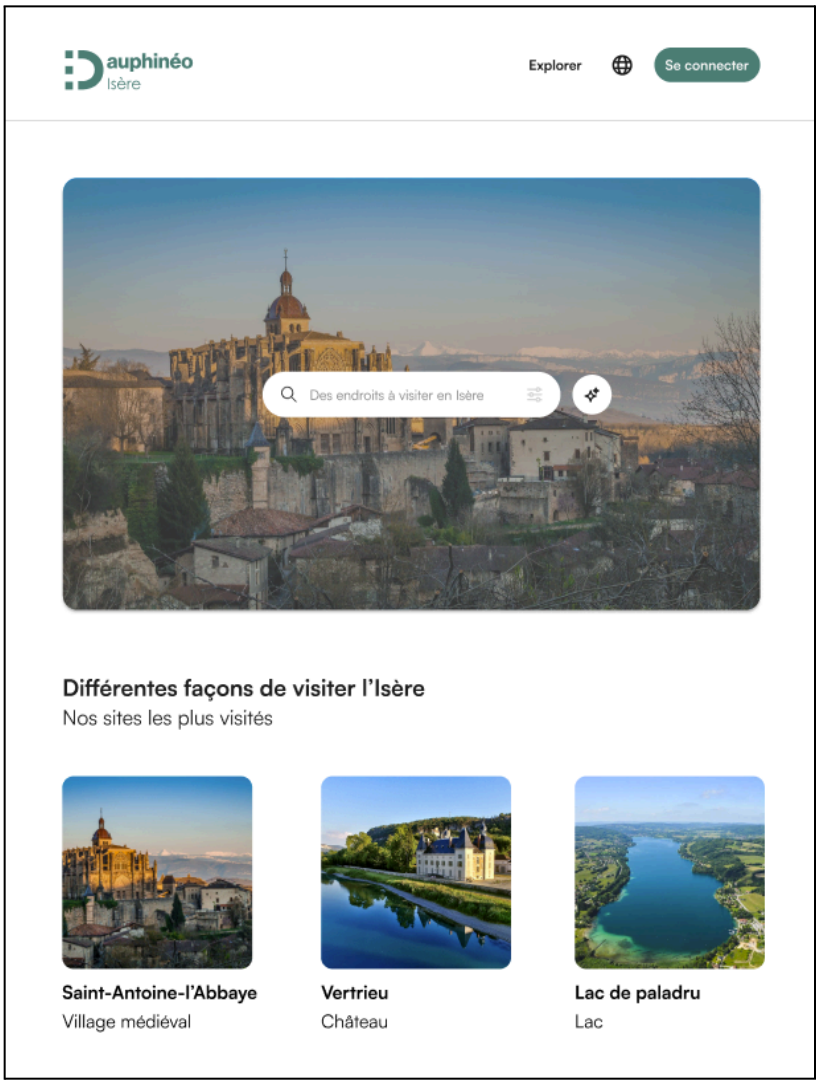
Le **header** s'intègre bien avec le reste de la page en utilisant une bordure de séparation simple. Les boutons respectent la charte graphique du site.

Le **fond blanc** de la page permet de mettre en avant les éléments de la page. Nous prévoyons également d'intégrer un dark mode pour le confort visuel.

L'élément principal, la **barre de recherche** est centrée sur les deux axes, sur un fond représentant un SPR.

L'icône ✨ pour le **questionnaire personnalisé** évoque l'expérience unique. Elle symbolise quelque chose de spécial ou d'exclusif.

Les **titres de section** et sous-titres sont suffisamment espacés, rendant donc le site épuré.



La section des **sites les plus visités** met en avant trois sites en parallèle, avec leurs photos, et sans trop de détails. Cela permet à l'utilisateur de directement savoir le SPR ciblé. Cela permet d'avoir un design en **grille**, **uniforme**, qui rend le site plaisant à regarder.

On évite au maximum la surcharge informationnelle sur cette page d'accueil, en laissant le **choix** à l'utilisateur. Voyons maintenant une page de SPR détaillée.

Page SPR détaillée

Lors du clic sur un des SPR les plus visités, voici ce qui s'affichera. Ceci est un **extrait**. La page complète se trouve en annexe (p.43).

La charte graphique est respectée et suit le reste du site. Le header est également identique, ce qui permet une **cohérence** et une uniformité sur toutes les pages du site.

Un bouton **Retour à l'accueil** permet à l'utilisateur de facilement revenir de là où il vient.

Le **titre** et le sous-titre sont clairement identifiables, permettant à l'utilisateur de savoir de quel SPR il s'agit.

Les **photos** s'affichent sous forme d'une grille libre, avec un pop-up pour afficher le reste des photos.

La **section principale** se démarque par un arrière-plan blanc cassé, ce qui permet de faire une distinction claire entre les informations principales et détaillées.

La section **informations** contient un paragraphe détaillant le SPR, avec une map permettant de localiser le site facilement.



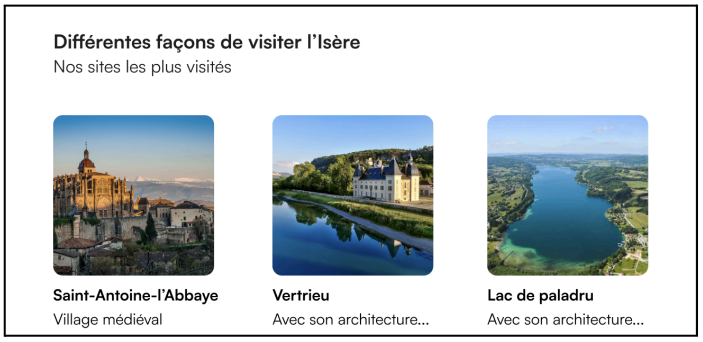
En dessous se trouve une fonction principale : la **réservation de guide**. Nous l'avons implémenté sous la forme d'un carrousel (défilement horizontal) affichant les différents guides proposés pour ce SPR. Un seul et unique bouton pour **réserver le guide** permet de révéler des informations complémentaires. Un petit badge permet de distinguer rapidement les guides officiels des guides amateurs.

Mise en oeuvre des besoins non-fonctionnels

Maintenant que nous avons une idée des besoins non-fonctionnels de nos utilisateurs, nous devons réfléchir aux manières de les **implémenter**, notamment en respectant les critères de **Bastien et Scapin**.

Guidage

Nous disposons, à travers de nos maquettes, d'une **organisation claire** des composantes, notamment les sites patrimoniaux remarquables (SPR) qui sont **regroupés** et présentés de manière **uniforme** et logique. Ce groupement visuel permet à l'utilisateur d'accéder aux fonctionnalités souhaitées sans difficulté de compréhension.



Dans notre projet, nos différentes maquettes en relation avec les différentes composantes de notre application web contiennent une **charte graphique cohérente**, de plus les différentes fonctionnalités et options sont toujours positionnés uniformément avec des couleurs identiques pour renforcer cet aspect de lisibilité.

Cohérence

Nos maquettes suivent une charte graphique cohérente. Les fonctionnalités et options sont uniformes, avec des couleurs standardisées pour améliorer la lisibilité.



Contrastes de luminosité

Les teintes “Seal Brown” (3F1F00), “Mint Cream” (EDF3EB), “Licorice” (201000) offrent un contraste suffisant, ce qui est favorable pour la lisibilité des textes sur ces fonds, ainsi que la lecture pour les personnes ayant des troubles visuels.

Cohérence visuelle

Les couleurs s’intègrent harmonieusement, ce qui renforce l’identité graphique unifiée que nous souhaitons partager. Cela garantit une expérience utilisateur harmonieuse en évitant les contrastes de couleurs trop agressifs sur l’ensemble de l’application.

Adaptabilité pour les zones interactives

Les teintes plutôt foncées “Seal Brown” (3F1F00) et “Licorice” (201000) sont idéales pour les zones interactives de navigation ou arrière plan pour les menus, tandis que les couleurs plutôt douces conviennent aux arrières plan de contenu ou certains éléments interactifs tels que les formulaires d’inscription ou connexion.

Signification des codes et dénominations

Au sein de notre projet Dauphinéo, nous disposons premièrement d'icônes explicites, chaque action ou fonctionnalité est accompagnée d'éléments visuels clairs tels que la barre de recherche qui dispose d’un icône explicite.

Focus sur les fonctionnalités principales

La barre de recherche est centrée sur une action clé : la **recherche**. Cela suit le principe de priorisation des fonctionnalités essentielles dans l’IHM.
L’icône permettant de répondre au questionnaire est également présente, offrant un accès rapide et direct à cette fonctionnalité clé. Cela contribue et facilite l’engagement de nos utilisateurs.

Icônes facilement identifiables

L’icône de recherche est universellement reconnue, ce qui renforce la compréhension et l’utilisabilité. De plus, le bouton permettant d’accéder à son profil ou de se connecter est également facilement identifiable.

Retour sur nos interfaces hommes-machine

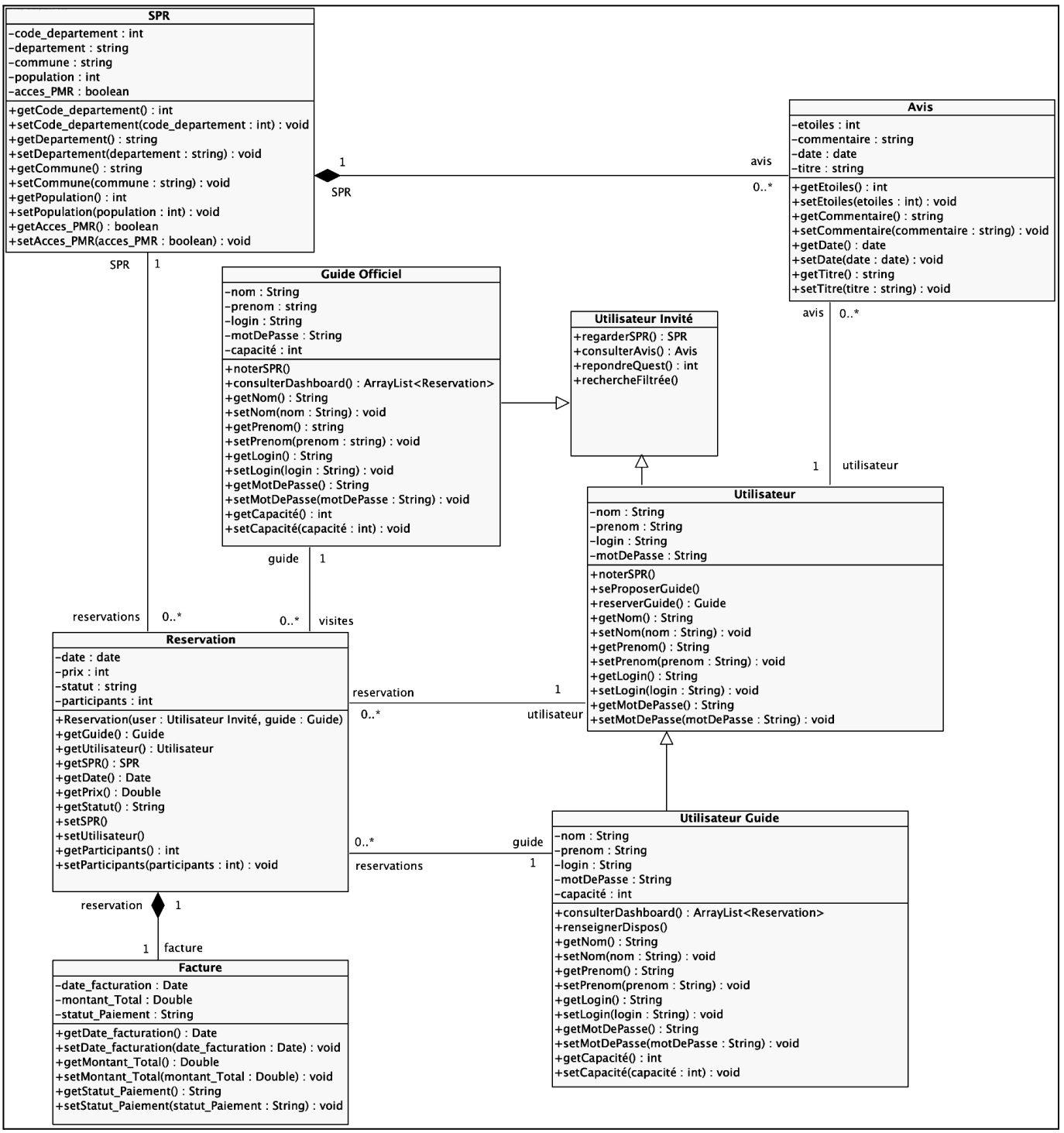
Pour garantir une interface utilisateur convenable, nous prévoyons de mettre à disposition d’utilisateurs test un questionnaire **SUS (System Usability Scale)** qui permet de mesurer la perception des utilisateurs sur l’utilisabilité de nos interfaces. Cette démarche a plusieurs objectifs

- Identifier les points forts et les axes d’amélioration de notre interface.
- Vérifier que le score moyen ait atteint ou dépasse 70, seuil indiquant une bonne expérience utilisateur.
- Ajuster les interfaces en fonction des retours obtenus.

Le questionnaire **SUS (System Usability Scale)** se compose de 10 affirmations, auxquelles les utilisateurs répondent sur une échelle de Likert allant de 1 (“Pas du tout d’accord”) à 5 (“Tout à fait d’accord”).

Architecture du code

Pour représenter l'architecture du code, nous avons décidé de passer par un **diagramme de classe**. Le voici :



Cette organisation du code nous permet de répondre aux besoins et fonctions de l'application et de réaliser efficacement ce qu'on doit faire.

Implémentation de la base de données

Voici le schéma (SLR) choisi pour implanter notre base de données :

Relations finales

- **UTILISATEUR**(id, nom, prenom, email, mot_de_passe, pseudo, verif_age, anonymise, dernier_contact, #type_utilisateur_id [FK→TYPE_UTILISATEUR(id)])
- **TYPE_UTILISATEUR**(id, libelle)
- **GUIDE**(#id [FK→UTILISATEUR(id), capacite_max) (Relation 0,1 avec UTILISATEUR : un guide est un utilisateur. Ici, id est à la fois PK et FK.)
- **LANGUE**(id, code_langue)
- **THEME**(id, nom_theme)
- **MOT_CLE**(id, nom_mot_cle)
- **DEPARTEMENT**(id, code_departement, nom_departement)
- **SITES_PATRIMONIAUX_REMARQUABLES**(id, #departement_id [FK→DEPARTEMENT(id)], nom_commune, population, is_pmr)
- **STATUT_RESERVATION**(id, libelle_statut)
- **STATUT_PAIEMENT**(id, libelle_statut)
- **JOUR_SEMAINE**(id, nom_jour)
- **RESERVATION**(id, date, heure, prix, #statut_id [FK→STATUT_RESERVATION(id)], #utilisateur_id [FK→UTILISATEUR(id)], #site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)], nombre_participants, taille_groupe, #theme_id [FK→THEME(id)], #langue_id [FK→LANGUE(id)])
- **AVIS**(id, titre, commentaire, note, date_publication, #utilisateur_id [FK→UTILISATEUR(id)], #site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)]) (CHECK(note BETWEEN 1 AND 5))
- **FACTURE**(id, date_facturation, montant_total, #statut_paiement_id [FK→STATUT_PAIEMENT(id)], #reservation_id [FK→RESERVATION(id)]) (Relation 1,1 avec RESERVATION : une réservation a une facture unique.)
- **DISPONIBILITE**(id, #jour_id [FK→JOUR_SEMAINE(id)], heure_debut, heure_fin, #guide_id [FK→GUIDE(id)], #site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)])
- **EVALUATION_GUIDE**(id, #guide_id [FK→GUIDE(id)], #site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)], note, commentaire) (CHECK(note BETWEEN 1 AND 5))
- **ACTION_TYPE**(id, nom_action)
- **DELAI_CONSERVATION**(id, table_reference, attribut_reference, delai_conservation, #action_id [FK→ACTION_TYPE(id)])
- **SUIVI_DONNEES**(id, entite_table, entite_id, debut_delai, date_suppression, #delai_conservation_id [FK→DELAI_CONSERVATION(id)])

Tables de liaison

- **GUIDE_LANGUE**(#guide_id [FK→GUIDE(id)], #langue_id [FK→LANGUE(id)], (guide_id, langue_id) [en PK composite])
- **SITE_THEME**(#site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)], #theme_id [FK→THEME(id)], (site_id, theme_id) [en PK composite])
- **SITE_MOT_CLE**(#site_id [FK→SITES_PATRIMONIAUX_REMARQUABLES(id)], #mot_cle_id [FK→MOT_CLE(id)], (site_id, mot_cle_id) [en PK composite])

En annexe (p. 40) se trouve le **Schéma Entité-Associations**.

Etude de faisabilité technique

Analyse des contraintes et des opportunités technologiques

Analyse des besoins techniques

Cette phase a pour objectif de préciser les exigences techniques nécessaires à la réalisation du projet. Elle consiste à étudier l'écosystème technologique existant et à identifier les contraintes associées à chaque composant, afin de sélectionner les technologies les plus appropriées pour répondre aux besoins du projet.

Evaluation des contraintes techniques

Front-End

Stack (version)	Justifications	Limitations techniques
React.js (18.3.1)	Permet un rendu dynamique et interactif des pages.	Maîtrise de React pour optimiser rendus et gestion des états. Pas de SSR/SSG (SEO faible), structure de fichiers non standard, dépendance à Recoil pour l'état global, absence d'animations complexes, bundles lourds pouvant impacter les performances.
Typescript (~5.6.2)	Fournit un typage statique pour réduire les erreurs et améliorer la maintenabilité du code.	Maîtrise des typages (statiques, génériques, interfaces) nécessaires. Transpilation coûteuse, incompatibilités avec certaines bibliothèques JS, typage any/unknown réduisant la sécurité, débogage complexe, surcharge de types, et problèmes de compatibilité avec les versions JS antérieures.
Recoil (0.7.7)	Simplifie le management des états partagés, tout en conservant une bonne intégration avec React.	Documentation limitée, performances affectées par des re-rendus inutiles, incompatibilité de Recoil avec d'anciennes versions de React, et gestion complexe des mises à jour des atomes.
Axios (1.7.9)	Simplifie les appels API. Gestion des promesses et des réponses JSON. Gestion des erreurs. Prise en charge de requêtes HTTP complexes Défini des en-têtes HTTP	Taille du bundle. Dépendance externe. Manque de fonctionnalités avancées dans certains cas.
Tailwind CSS (3.4.13)	Facilite une intégration rapide et permet de styliser des composants avec des classes utilitaires.	Lisibilité et maintenabilité faibles, classes redondantes et longues, absence de composants prédéfinis gratuits, et forte dépendance aux outils de build.

Back-End

Stack	Justifications	Limitations techniques
Laravel (11)	Offre une structure MVC, des outils intégrés pour l'authentification, la validation, et les requêtes API. Permet d'éviter le développement procédural souvent associé au PHP natif.	Laravel : Consommation élevée de ressources, ralentissements avec grandes bases ou requêtes non optimisées, courbe d'apprentissage exigeante (middleware, jobs).
PostgreSQL (15.9)	Connu pour sa robustesse, sa conformité ACID, et ses fonctionnalités avancées (ex. gestion des relations complexes, JSON supporté).	Les conflits dus à des modifications simultanées peuvent ralentir les performances, surtout avec des transactions lourdes. PostgreSQL est aussi gourmand en mémoire et CPU avec des bases volumineuses ou des requêtes complexes.

Tests

Stack	Justifications	Limitations techniques
Pest (3.6)	Adapté pour Laravel. Automatisation des tests unitaires, fonctionnels et d'intégration avec la base de données.	Performance limitée sur de grands volumes de données Exécution gourmande en ressources pour les tests Tests back-end et logiques métier Limité pour tester des composées spécifiques
Cypress	Cypress est adapté pour JavaScript, React, Laravel, Tailwind, TypeScript et peut tester les changements d'état produit par recoil	Pas conçu pour le test d'API. Applications multi-pages : Gestion difficile des redirections ou des sessions entre plusieurs pages. Pas de prise en charge multi-navigateurs.

Administration système

Stack	Justifications	Limitations techniques
Hyperviseur ASSR	Séparation des responsabilités Architecture évolutive Flexibilité de gestion : sauvegarde, maintenance indépendantes Sécurité renforcée : restriction de l'accès à la base de données.	Peut entraîner un surcoût de ressources par rapport à l'exécution de services sur des machines physiques. Latence réseau entre la VM web et la VM BDD Synchronisation des sauvegardes
VM BDD (PostgreSQL)	/	Limites des ressources allouées Surcharge de virtualisation Latence et bande passante réseau Risque de panne de l'hyperviseur Volume de données massifs et requêtes complexes peuvent rencontrer des limites de performance
VM Web (Apache et PHP)	/	Performance : dépendance des ressources allouées par l'hyperviseur, possible ralentissement si forte charge La surcharge liée à la virtualisation peut affecter le traitement des requêtes HTTP et l'exécution des scripts PHP

Infrastructure existante

L'infrastructure actuelle dispose de :

Ressources matérielles

Serveur

- Un serveur nommé **ASSR** qui fait office d'hyperviseur pour faire tourner des machines virtuelles
- Deux processeurs de 4 cœurs chacun
- 128 Go de RAM
- 1 volume de stockage HDD de 2.7 To en RAID 1 (mirroring)

Machines virtuelles

- **Machine virtuelle 1** (192.168.14.115) qui fait office de serveur web
  - **Espace disque** disponible :

```
root@debian:~# df -h
Filesystem                   Size  Used Avail Use% Mounted on
udev                         960M     0  960M   0% /dev
tmpfs                        197M  536K  197M   1% /run
/dev/mapper/debian--vg-root  7.5G  3.0G  4.2G  42% /
tmpfs                        984M  1.1M  983M   1% /dev/shm
tmpfs                        5.0M     0  5.0M   0% /run/lock
/dev/sda1                    455M  156M  275M  37% /boot
tmpfs                        197M     0  197M   0% /run/user/1000
tmpfs                        197M     0  197M   0% /run/user/0
```

  - **Processeur** : 1 processeur de 1 cœur
  - **Mémoire** : 1.9 Go de RAM au total - Actuellement 1.3 Go disponibles

```
root@debian:~# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       319Mi       1.3Gi        28Mi       482Mi       1.6Gi
Swap:          975Mi          0B       975Mi
```

- **Machine virtuelle 2** (192.168.14.215) qui fait office de base de données
  - **Espace disque** disponible :

```
root@debian:~# df -h
Filesystem                   Size  Used Avail Use% Mounted on
udev                         960M     0  960M   0% /dev
tmpfs                        197M  528K  197M   1% /run
/dev/mapper/debian--vg-root  7.5G  3.0G  4.2G  42% /
tmpfs                        984M  1.1M  983M   1% /dev/shm
tmpfs                        5.0M     0  5.0M   0% /run/lock
/dev/sda1                    455M  156M  275M  37% /boot
tmpfs                        197M     0  197M   0% /run/user/0
```

L'espace disponible sur la partition "**/**" (4.2 Go) est suffisant pour héberger une base de données de petite à moyenne taille comme PostgreSQL. Dans le cas d'une croissance importante de la base de données, il serait prudent d'étendre l'espace volume logique sd6.

- **Processeur** : 1 processeur de 1 cœur
- **Mémoire** : 1.9 Go de RAM au total - Actuellement 1.3 Go disponibles

```
root@debian:~# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       287Mi       1.3Gi        19Mi       477Mi       1.6Gi
Swap:          975Mi          0B       975Mi
```

Postes de travail

- Postes fixes de l'IUT ou nos PC portables personnels.

Ressources logicielles

- **Système d'exploitation** : Linux (Debian sans interface graphique)
- **Logiciels installés sur les serveurs**
  - **Serveur web** : Apache2 et PHP 8.3.13
  - **Système de gestion de base de données** : PostgreSQL (15.9)
- **Logiciels de sécurité** : ufw (0.36.2), fail2ban
- **Outils de versionning** : Git (2.39.5)

Tests de l’infrastructure

Tests de connexion à la BDD et du script

Ce test nous permet de vérifier la connexion à la base de données ainsi que l’affichage des données lors de l'exécution du script.

Pour autoriser la communication entre la VM1 (serveur web) et la VM2 (BDD), nous configurons ainsi le fichier **pg_hba.conf** de la VM2 :

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
host    all             mdcb            192.168.14.115/24       scram-sha-256
```

Pour vérifier la connexion à la base de données, nous avons créé un fichier **test.php** dans **/var/www/html/** de notre VM1. Voici le code présent dans ce fichier :

```
<?php
$host='192.168.14.215';
$port='5432';
$dbname='test';
$user='mdcb';
$password='xb3J%PSBz1';

$conn = pg_connect("host=$host port=$port dbname=$dbname user=$user password=$password");

if (!$conn) {
    echo "La connexion à la base de données a échoué.";
} else {
    echo "Connexion réussie";
}

$sql = "SELECT * FROM users";
$result = pg_query($conn, $sql);

if ($result) {
    while ($row = pg_fetch_assoc($result)) {
        echo "ID: " . $row['id'] . " - Nom: " . $row['name'] . "\n";
    }
} else {
    echo "Erreur lors de l'exécution de la requête.\n";
}

pg_close($conn);
?>
```

Dans un premier temps, nous effectuons la connexion avec la BDD avec **PostgreSQL**, en utilisant **pg_connect**. On vérifie que la connexion à la BDD à réussi. Si ce n’est pas le cas, on affiche un message d’erreur.

Par la suite, nous effectuons une requête basique “SELECT * FROM users”, afin d’afficher tous les utilisateurs de la table “users”. Afin que la requête puisse afficher les données correspondantes, nous vérifions que le résultat de la requête existe. Dans ce cas, on affiche toutes les lignes, sinon on indique un message d’erreur concernant l’exécution de la requête.

On teste notre fichier avec la commande **php test.php** :

```
root@debian:/var/www/html# php test.php
Connexion réussieID: 1 - Nom: Alice
ID: 2 - Nom: Bob
ID: 3 - Nom: Charlie
ID: 4 - Nom: Maelys
```

Les tests de connexion à la BDD ainsi que l’affichage des données est donc réussi.

Sécurité et conformité

Configuration système et serveur

Il faut une bonne **gestion des permissions** :

- Les fichiers web (HTML, PHP, CSS) doivent être lisibles par le serveur web (`www-web`)
  - `chown -R root:www-data /var/www/html && chmod -R 750 /var/www/html`
- Cela garantit que seul le propriétaire (`root`) et le groupe (`www-data`) peuvent accéder aux fichiers.

Pare-feu (ufw)

Il faut autoriser uniquement les ports nécessaires :

```
ufw allow 22/tcp
ufw allow 443/tcp #HTTPS
ufw enable
ufw allow from 192.168.14.115 to any port 5432
```

Fail2Ban

Pour bloquer les tentatives de connexion malveillantes, on doit configurer le fichier : `/etc/fail2ban/jail.conf` :

```
port     = https
maxretry = 3
```

Sécuriser la base de données

Pour cela, on crée deux rôles : un **lecteur** pour la partie client, et un **éditeur** pour le serveur.

- Donner les droits de SELECT au lecteur
- Donner les droits de SELECT, INSERT, DELETE et UPDATE à l'éditeur

Authentification SSH par clé publique

- Ajouter une clé publique pour les connexions SSH

Environnement de travail

Installation de la librairie React et du framework Laravel

Nous avons installé la librairie React dans un dossier **frontend** à la racine du projet. L'utilisation du **bundle Vite** permet de mettre en place l'environnement de travail frontend efficacement. En effet, Vite comprend plusieurs packages : **React** et **Typescript**. Pour lancer l'application côté client, nous devons installer les **node_modules** nécessaires avec **npm** et exécuter la commande *npm run dev*.

Dans un dossier **backend** à la racine du projet, nous avons installé Laravel avec le gestionnaire de dépendances **composer**. Nous avons ensuite configuré le fichier **.env** de Laravel afin que ce dernier puisse interagir avec la base de données. Pour rappel, Laravel fait office d'API afin que les modèles soient sérialisées en JSON. Pour faciliter le développement, nous allons mettre en place une **documentation API** avec **OpenAPI** (package l5-swagger).

Organisation de travail

Pour permettre aux développeurs de travailler sur l'application sans risquer de perturber **l'environnement de production**, il est nécessaire de mettre en place un **environnement de développement distinct**. Ici l'environnement de production correspond aux VM distantes. Pour rappel, la VM 1 fait office d'hébergement de notre application et la VM 2 fait office de base de données. Afin que les membres de l'équipe puissent travailler en local, il est plus judicieux d'effectuer une copie de la base de données distante sur les machines locales.

Chaque fonctionnalité terminée sera poussée sur GitLab.

Déploiement du projet sur la VM 1 - serveur web

Une fois les fonctionnalités poussées sur le répertoire GitLab, nous aimerions mettre en place un déploiement CI/CD automatisé sur notre VM 1. Dans notre cas, la pipeline effectuera un **git pull** sur la machine distante.

Analyse de l’effectué

Bilan et retours sur les réalisations

Analyse de l'effectué et du restant a faire

Accompli

1. **Cadrage du projet** :
    - Retour sur les notions de contraintes et de risques. Nous avons choisi de condenser autant que possible les notions de contraintes et de risques, ce qui nous a permis de simplifier les informations en nous appuyant sur les détails déjà abordés lors de la première phase
    - Nous avons réalisé une analyse rétrospective des risques identifiés au cours de la première phase, qu'ils se soient matérialisés ou non.
2. **Expression des besoins** :
    - Identification et revue des besoins non-fonctionnels de notre application en se basant sur nos personas notamment en matière d'IHM (*Interaction Homme Machine*).
3. **Modélisation des besoins** :
    - Finalisation des maquettes.
    - Création des diagrammes BPMN.
    - Élaboration du modèle de données à l'aide d'un *SEA*.
    - Conception du plan du site.
4. **Réalisation** :
    - Étude de faisabilité technique complète.
    - Choix des technologies justifiées : React.js, TypeScript, Tailwind CSS pour le front-end ; Laravel et PostgreSQL pour le back-end.
    - Mise en place de l'environnement de travail.
5. **Architecture logicielle** :
    - Spécifier les interactions entre les modules front-end et back-end, notamment pour la gestion des recommandations et des réservations.
6. **Architecture réseau** :
    - Identifier les besoins en termes de connectivité et de sécurité.
    - Décrire la configuration des serveurs et les protocoles de communication.
    - Préciser la gestion des bases de données et l'accès aux API externes.

Restant à réaliser

1. **Développement de l'application**
    - Développement principal
    - Tests
2. **Rendu troisième phase + soutenance**

Annexes

Documents et références annexes

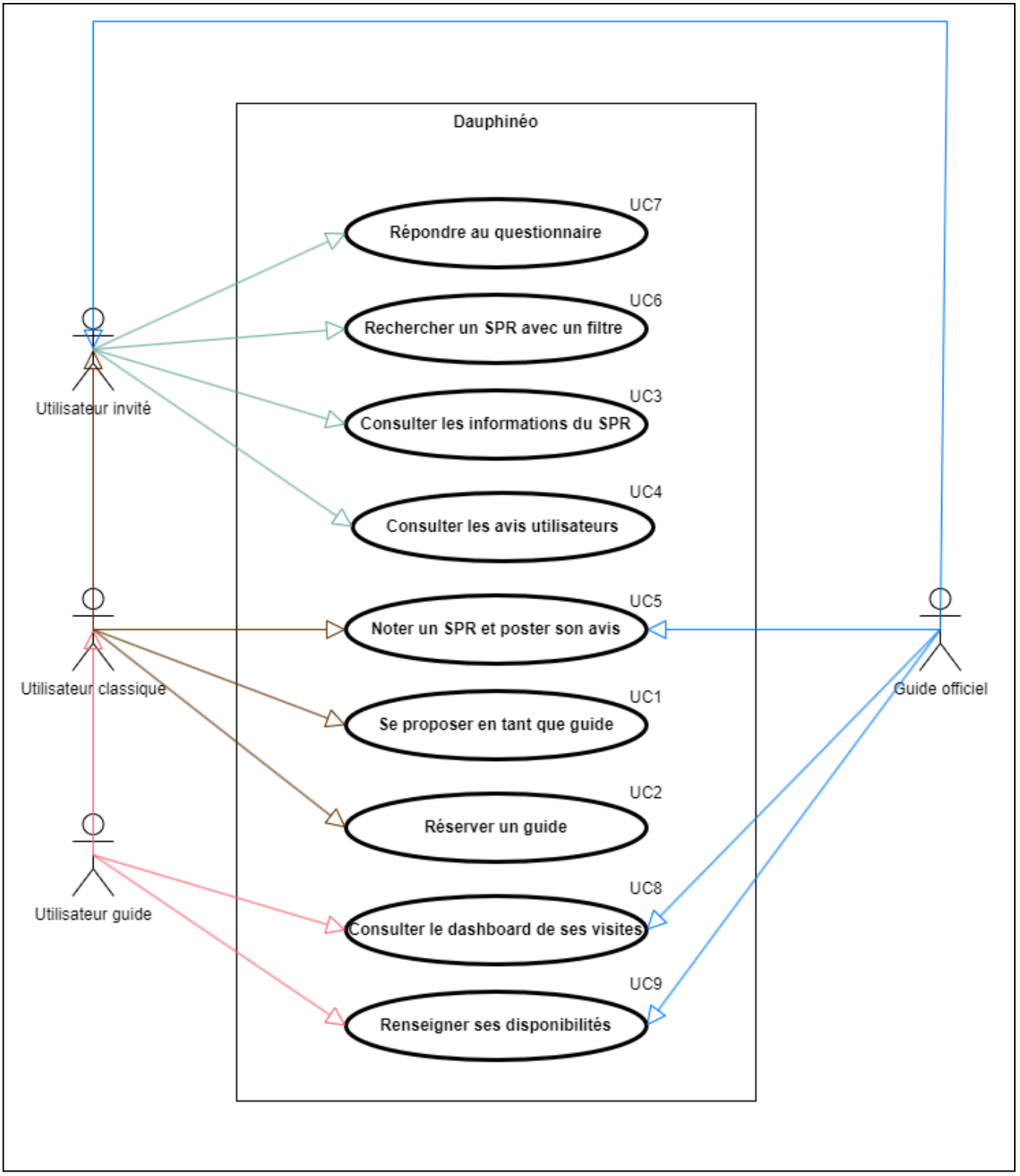
Matrice RACI

R : Responsible **A** : Accountable **C** : Consulted **I** : Informed

Dauphinéo	Damien	Timothé	Naël	Lilya	Maëlys	Sarah	Axel
Dossier de Conception							
Diagramme de cas d'utilisation	C	C / I	R	C / I	C / I	C / I	**A** / R
Diagramme Classe Métier	I	I	I	I	I	R	**A** / R
Diagramme Objet	I	I	I	I	I	**A** / R	R
Diagramme Séquence	I	I	I	I	I	R	**A** / R
Gestion de projet							
Cadrage du projet	C	**A** / R	R	I	I	I	I
Communication							
Création du logo et de la charte graphique	C	C / I	C / I	C / I	**A** / R	I	C / I
Pitch du 18/10	C / I	C / I	C / I	C / I	C / I	C / I	**A** / R
Soutenance 1 (organisation)	**A**	R	R	C	C	C	C
Soutenance 2 (organisation)	**A**	C	C	R	C	C	R
Soutenance 3 (organisation)	**A**	C	C	C	R	R	C
Relecture des dossiers avant rendu	**A** / R	I	I	I	I	I	I

Dauphinéo	Damien	Timothé	Naël	Lilya	Maëlys	Sarah	Axel
Anglais							
Pitch	R / C	**A** / R	R	R	R	R	R
IHM							
Persona	**A**	C / I	I	R	I	R	R
Maquettes	R	I	I	**A**	R	**A**	C / I
Réalisation							
Développement	**A** / R	R	R	R	R	R	R
Tests	**A** / R	C / I	C / I	C / I	C / I	C / I	C / I
Bases de données	I	C / I	C / I	**A** / R	**A** / R	R	I
Administration système	I	I	I	**A** / R	**A** / R	I	I

Diagramme de cas d’utilisations



Répartition des rôles (annexe)

Pour la répartition des rôles, nous avons choisi d'utiliser une matrice RACI afin de clarifier les responsabilités de chaque membre de l'équipe. La matrice RACI nous permet de définir qui est **Responsable** (R), **Accountable** (A), **Consulté** (C), et **Informé** (I) pour chaque tâche et livrable du projet.

Pour détailler cela plus simplement, nous nous sommes répartis les rôles de cette manière :

Damien

- Chef de projet
- S'occupe des interfaces (front-end)
- Contribue au code

Timothé

- S'occupe de la gestion de projet et de l'anglais
- Contribue au code

Axel

- S'occupe de la communication, de l'IHM, du BPMN et de l'UML
- Contribue au code

Naël

- S'occupe de la gestion de projet, de l'IHM et de l'UML
- Contribue au code

Lilya

- S'occupe de l'IHM, des bases de données et de l'administration
- Contribue au code

Sarah

- S'occupe de l'IHM, de l'UML et de l'anglais
- Contribue au code

Maëlys

- S'occupe de l'administration système, des applications (front-end et backend) et des bases de données
- Contribue au code

Conduite du projet (annexe)

Annexe et version condensée de la méthode **QQOQCP**, disponible en version détaillée dans le document de la phase 1.

Les acteurs (Qui ?)

La maîtrise d'ouvrage

- Les guides (officiels ou amateurs)
- Les SPR (associations, mairies, etc...)
- Les visiteurs passionnés et curieux du patrimoine
- Comité de **pilotage** (externe) : L'équipe pédagogique

La maîtrise d'oeuvre (équipe projet)

- Aravena Bravo Damien (**chef de projet**)
- Bendris Naël
- Jacob Timothé
- Deschaux-Beaume Maëlys
- Benkheira Lilya
- Sekma Sarah
- Tierrie Axel

Quoi ?

- Disposer de fiches d'informations détaillées
- Permettre des recommandations personnalisées sur la base d'un questionnaire
- Permettre de réserver un guide local spécialisé via l'application web
- Munir l'application d'un espace interactif, avec la possibilité de poster des avis

Supports de communication (Où et Comment ?)

Interne (Entre l'équipe de projet)

- Discord
- Réunion régulières (Discord et à l'IUT)
- Réseaux sociaux
- Notion

Externe (L'équipe projet à comité de pilotage)

- Mail
- Séances régulières
- Issues GitLab

La planification (Quand ?)

Rendu du deuxième dossier (modélisation)

- Rendu le **11 décembre 2024** à 12h
- Estimation : 32h de travail commun
- 7 après-midi bloquées - le reste en autonomie

Soutenance sur le deuxième dossier

- Passage le **13 décembre 2024**
- Estimation : 2h

Rétrospective sur la première présentation orale (anglais)

- Passage le **18 décembre 2024**
- Estimation : 2h
- Réunions en autonomie la semaine du 16 décembre

Rendu du code + soutenance et démonstration

- Échéance le **24 janvier 2025**
- Estimation : 108h de travail (3 semaines)
- Travail en autonomie. Possibilité d'alterner entre présentiel et distanciel

La coordination

Chaque membre de l'équipe à un rôle et des tâches définies, ce qui rend le déroulement du projet plus fluide. Cela permet également de mettre en avant les membres plus expérimentés dans leur domaine, afin de les assigner à la bonne tâche.

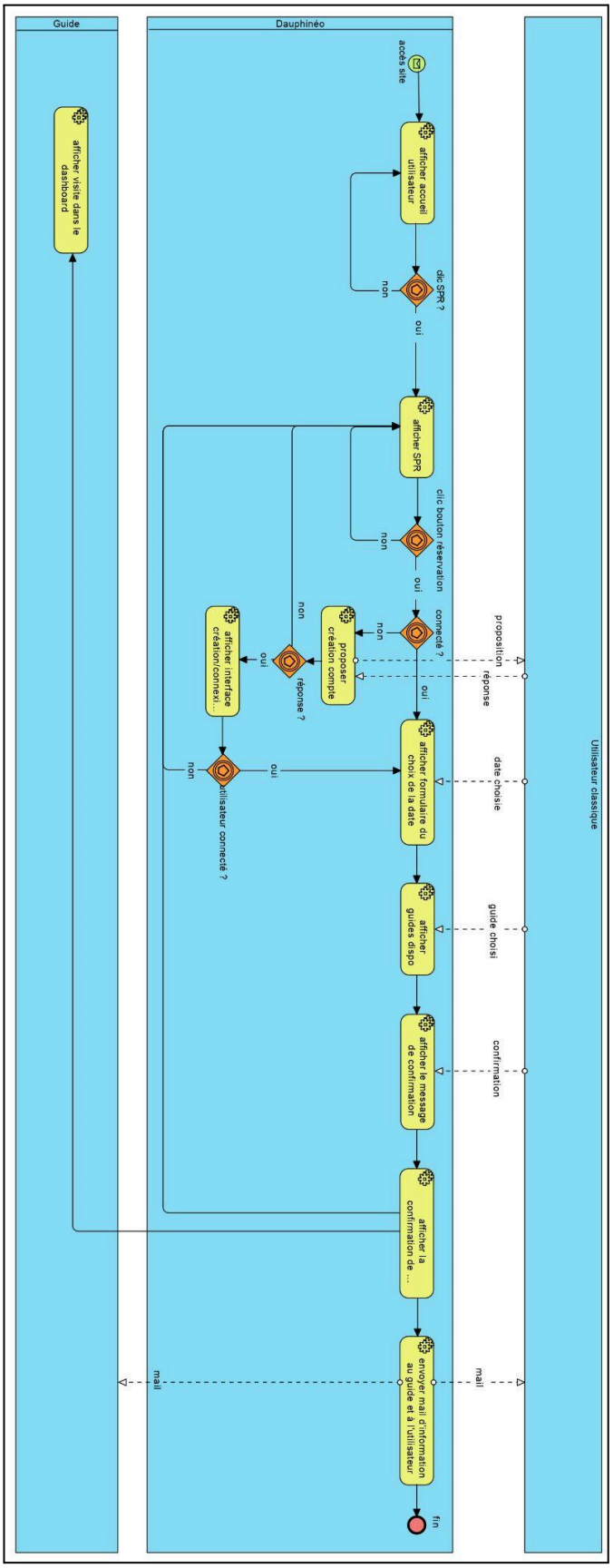
Pour cela, nous utilisons des outils de travail collaboratifs, tels que **Notion** avec un tableau Kanban, pour faciliter l'organisation. Il est également important de mentionner l'utilisation de la matrice RACI lors de cette phase de cadrage du projet.

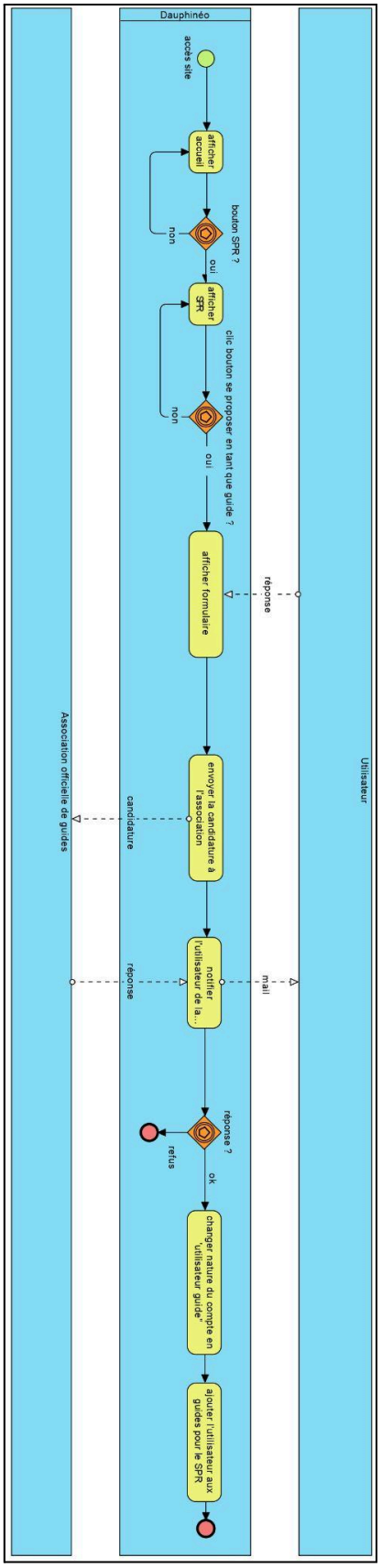
Le pilotage et la surveillance

Le projet sera piloté par les demandeurs, et cadré par les différents rendus et attendus. Les séances régulières avec les coaches nous permettent de suivre efficacement l'avancement, et de s'assurer que l'on vise toujours bien l'objectif.

Nous organisons également des réunions d'équipe afin de mettre au point l'avancement.

Diagramme BPMN - 1





Sitemap

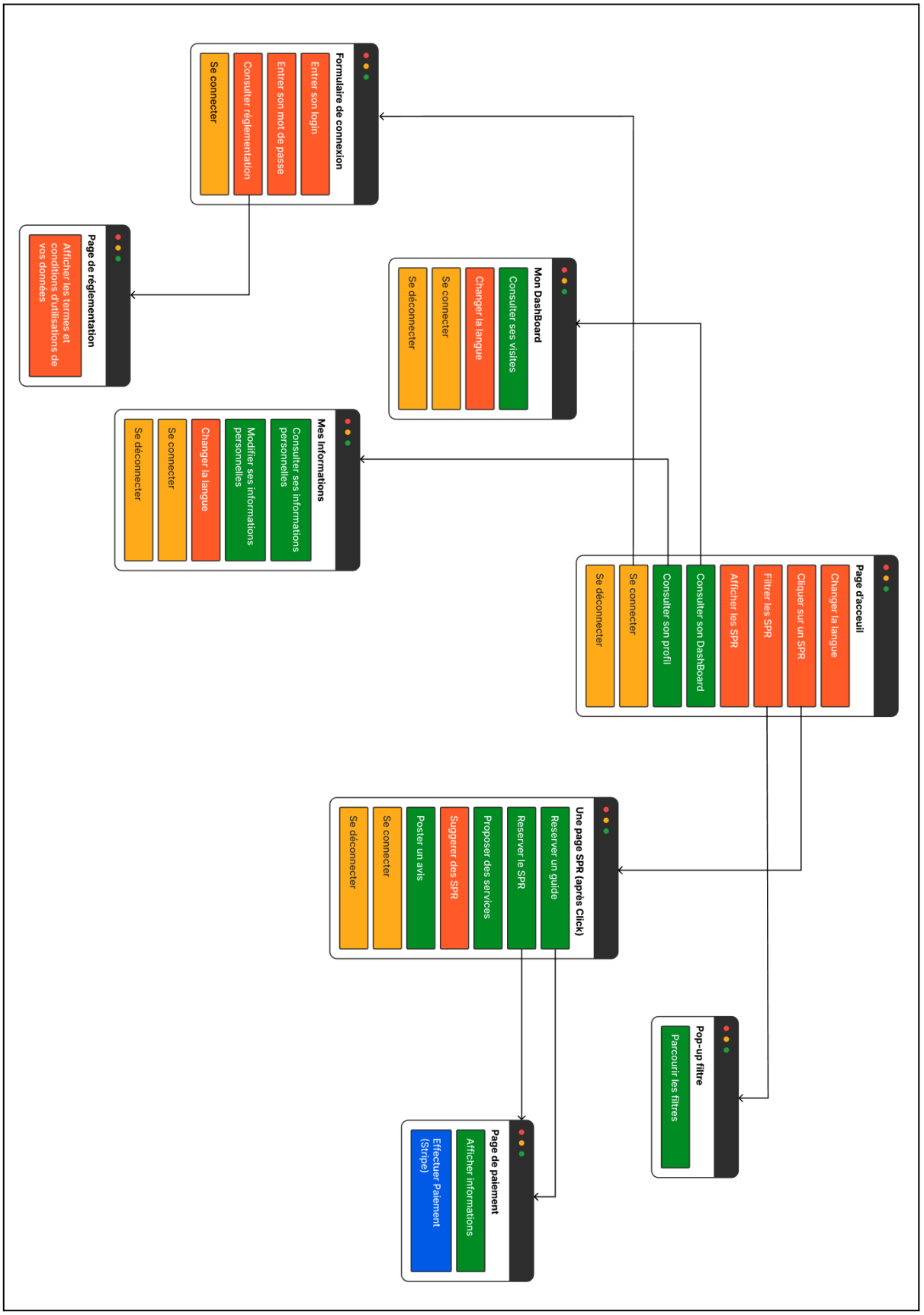
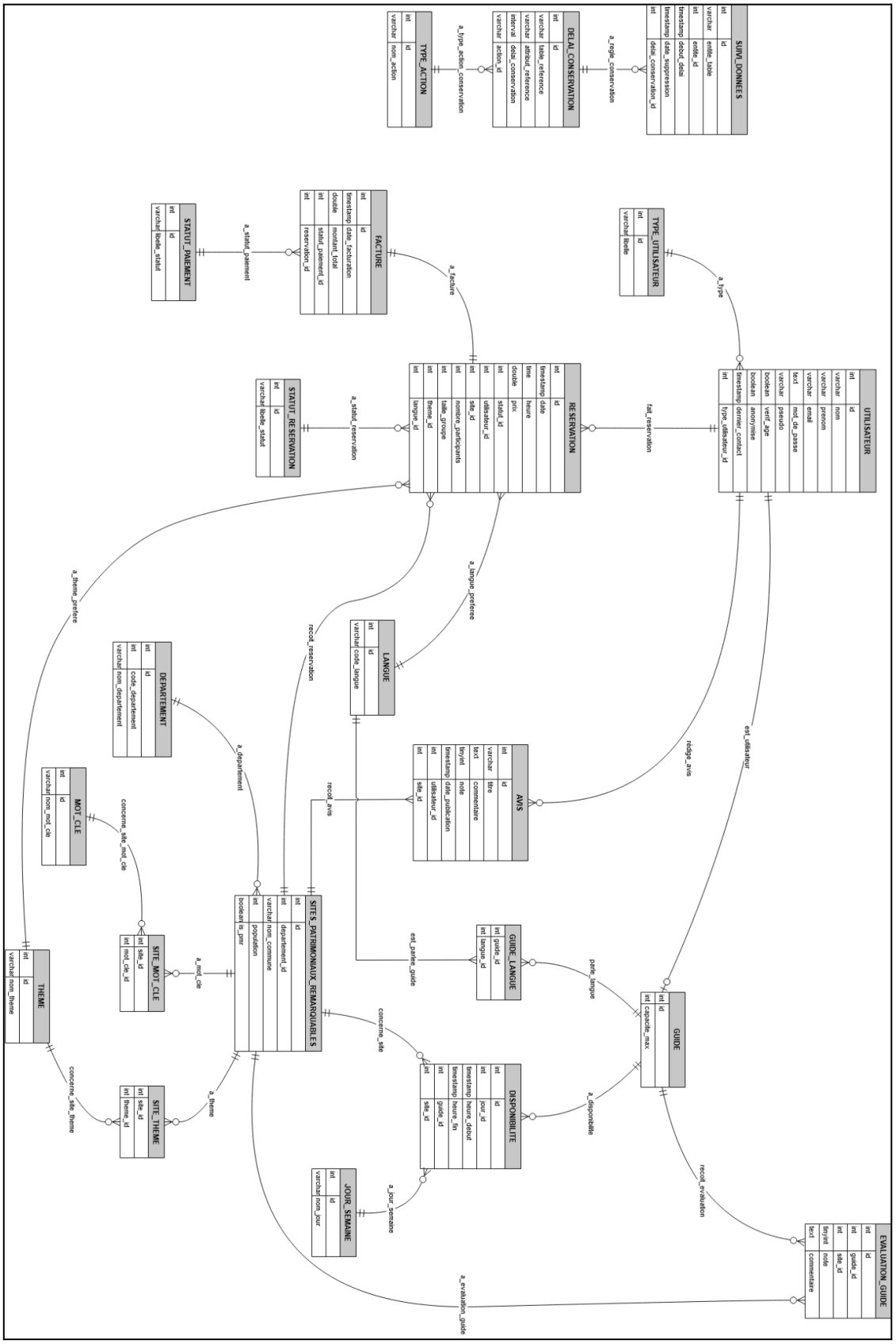
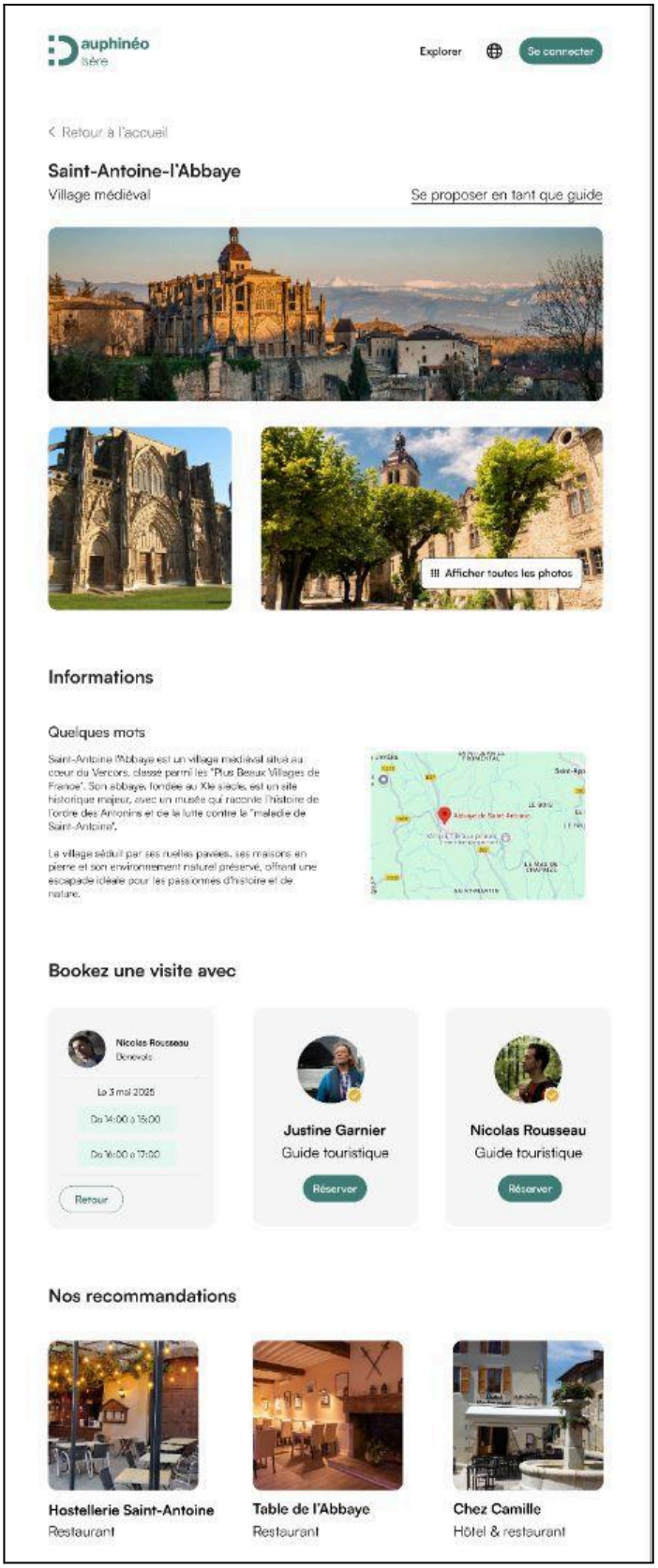
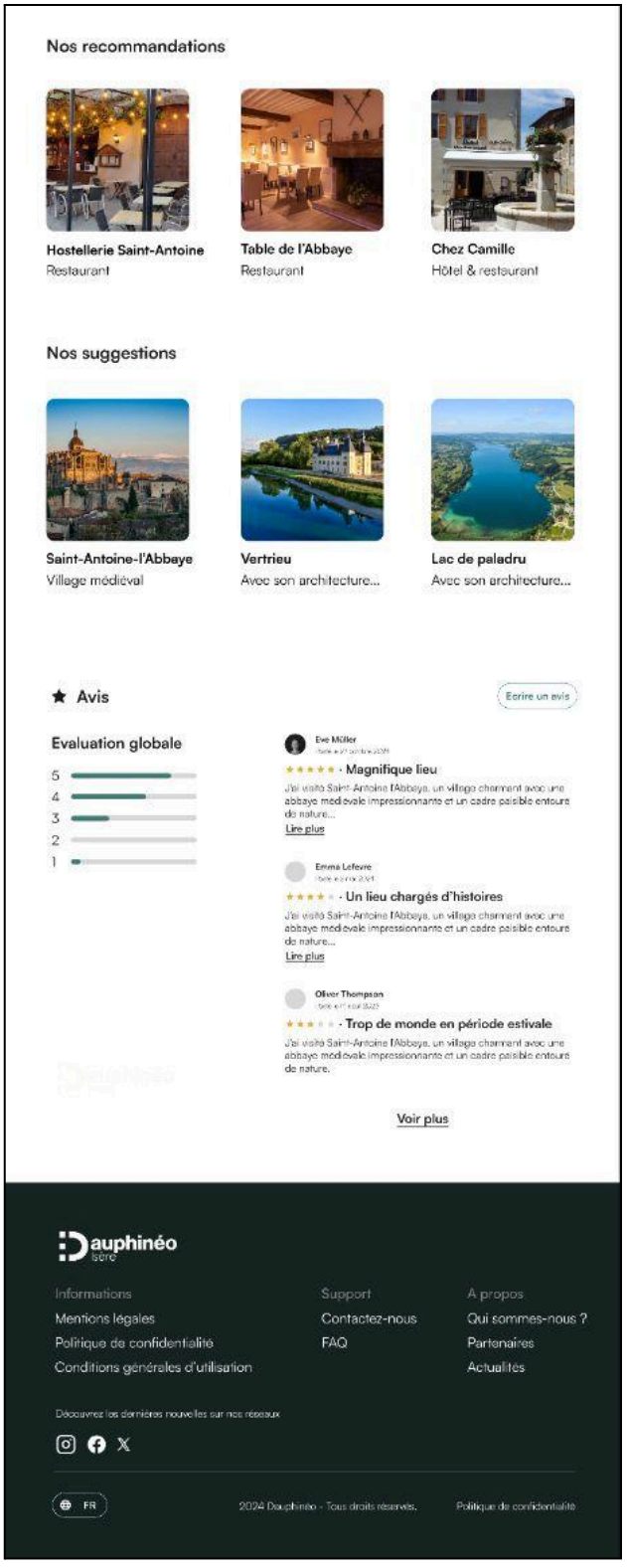

Formulaire de connexion
Entrer son login
Entrer son mot de passe
Consulter réglementation
Se connecter
Page de réglementation
Afficher les termes et conditions d'utilisations de vos données
Mon DashBoard
Consulter ses visites
Changer la langue
Se connecter
Se déconnecter
Mes Informations
Consulter ses informations personnelles
Modifier ses informations personnelles
Changer la langue
Se connecter
Se déconnecter
Page d'acceuil
Changer la langue
Cliquer sur un SPR
Filtrer les SPR
Afficher les SPR
Consulter son DashBoard
Consulter son profil
Se connecter
Se déconnecter
Une page SPR (après Click)
Reserver un guide
Reserver le SPR
Proposer des services
Suggerer des SPR
Poster un avis
Se connecter
Se déconnecter
Pop-up filtre
Parcourir les filtres
Page de paiement
Afficher informations
Effectuer Paiement (Stripe)

Schéma Entités-Associations

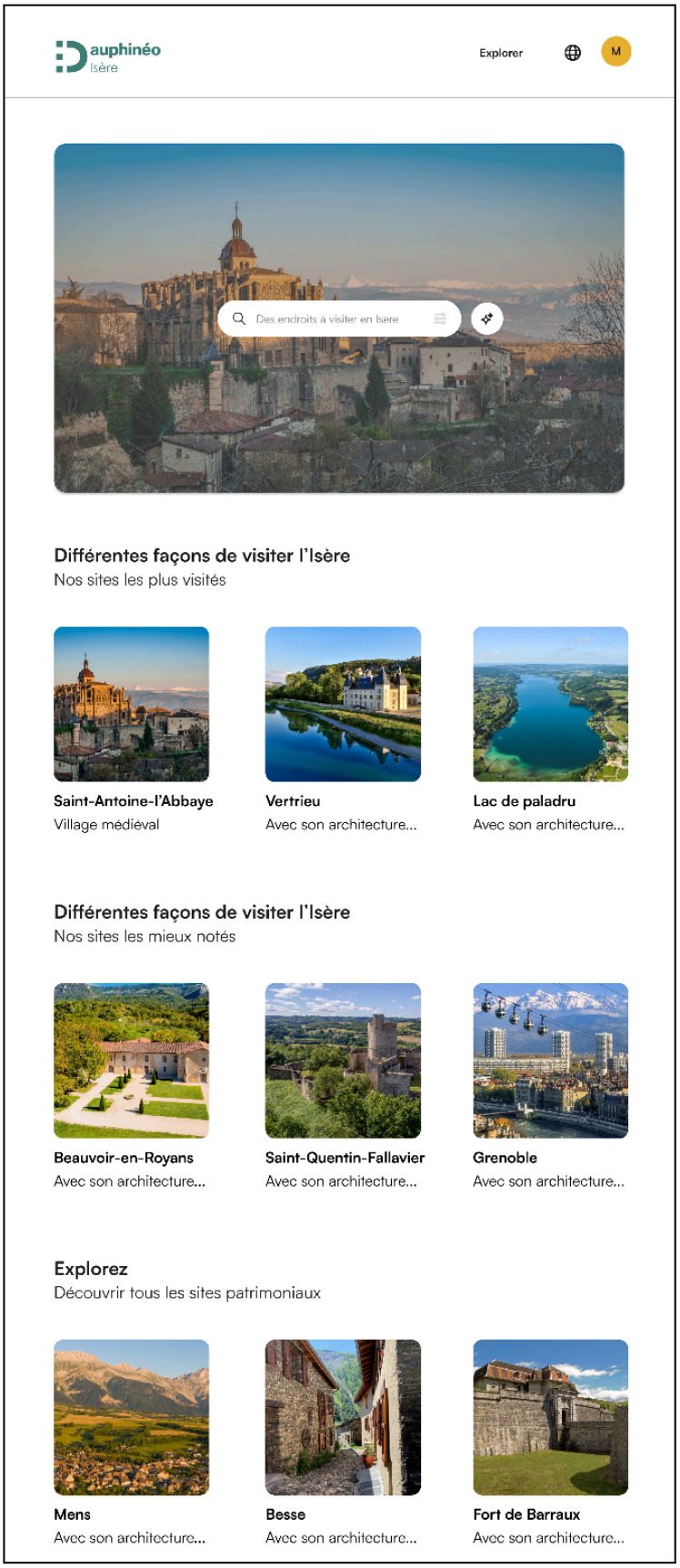


Maquettes - annexe

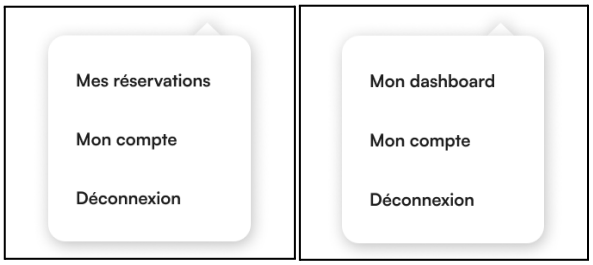




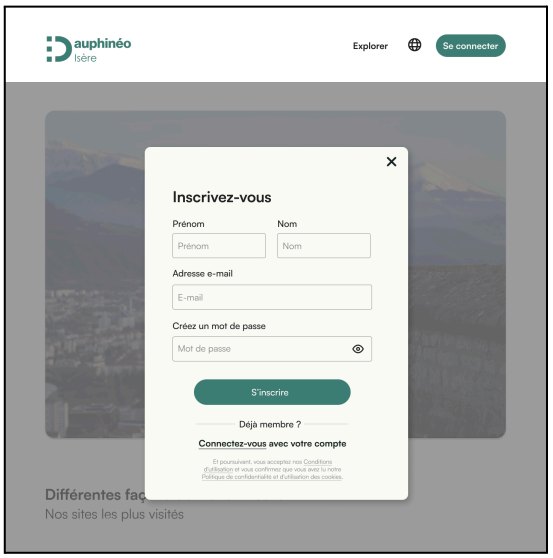
Maquette - Page Type SPR

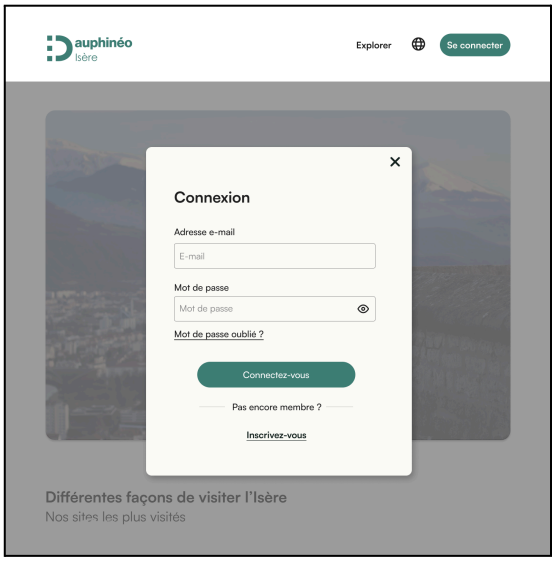


Page d'accueil - Connecté
Sans footer

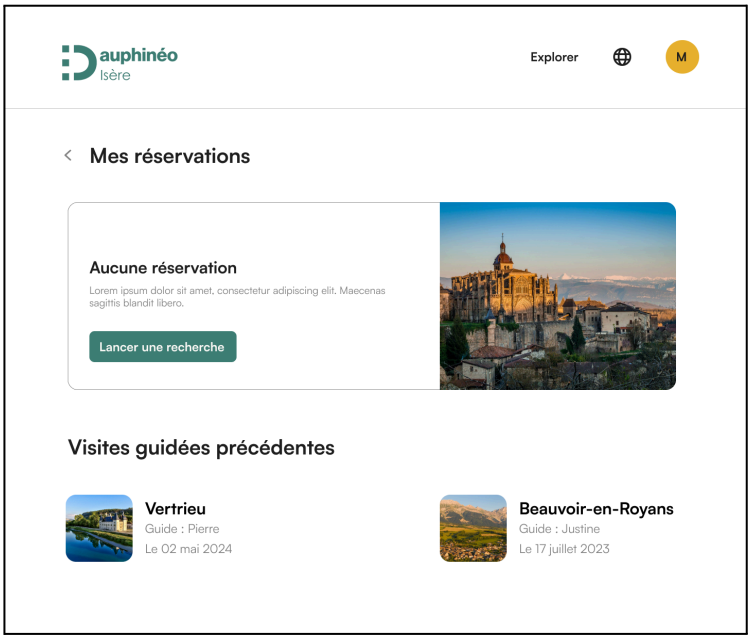


Diverses Pop-Ups

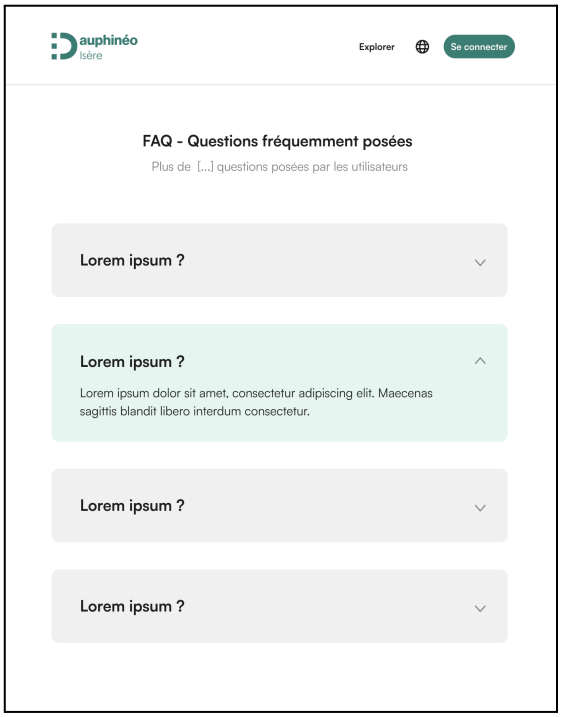




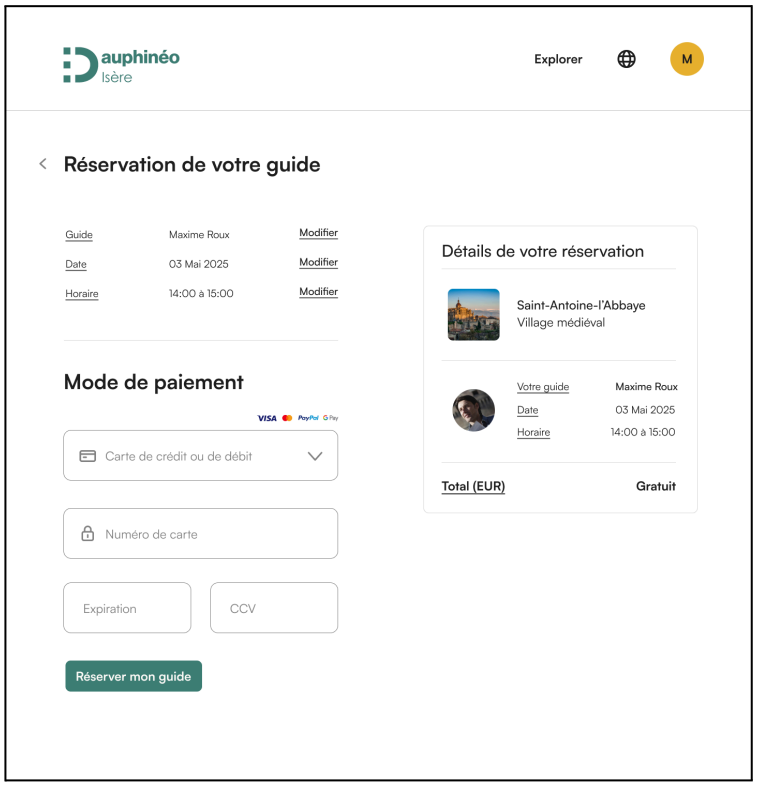
Pop-Ups inscription/connexion



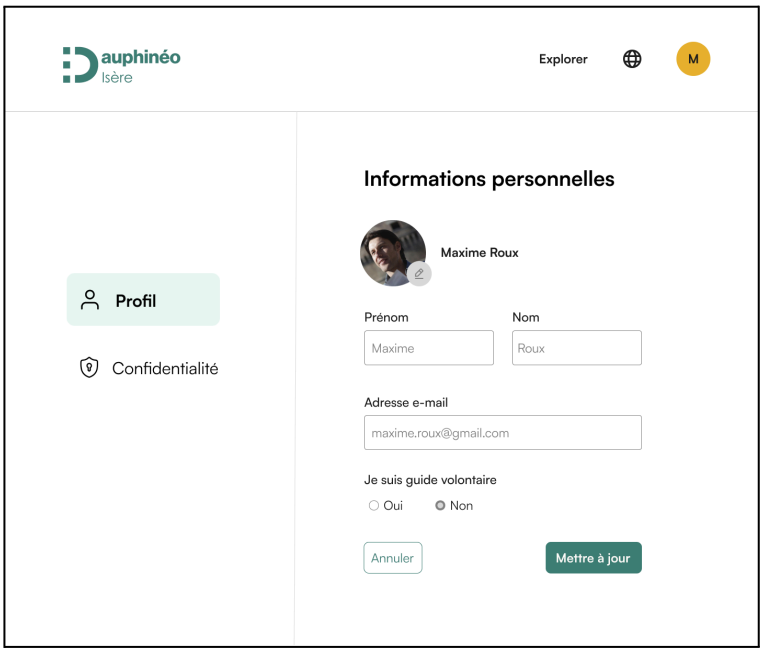
Mes réservations (avec historique)



Page des FAQ



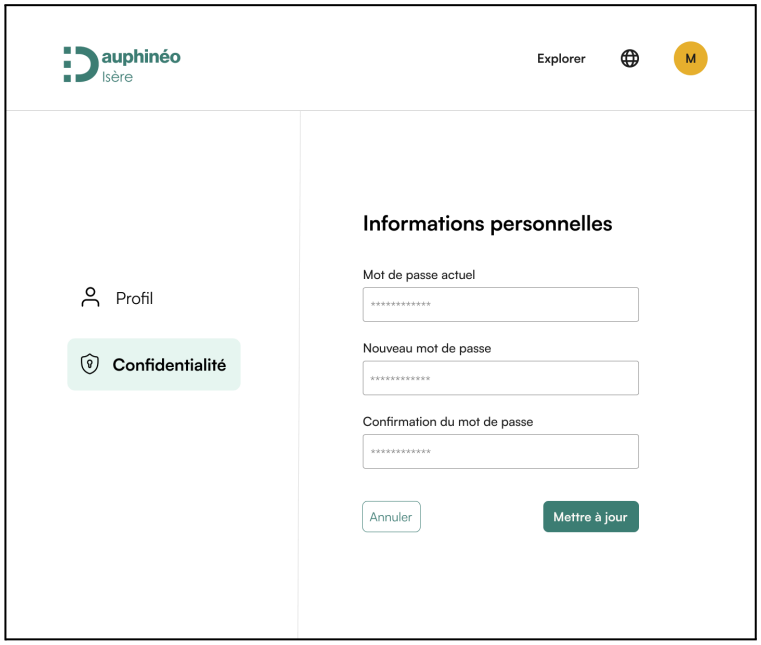
Page de réservation de guide



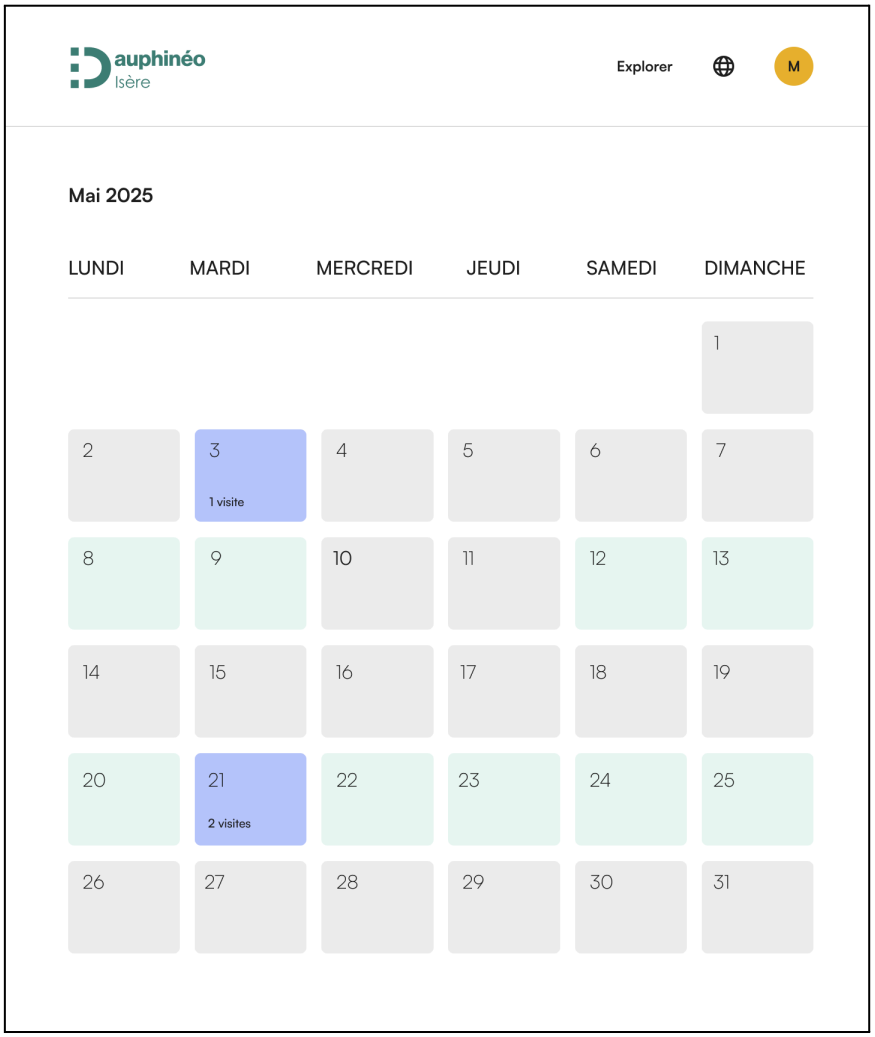
Page profil - utilisateur connecté



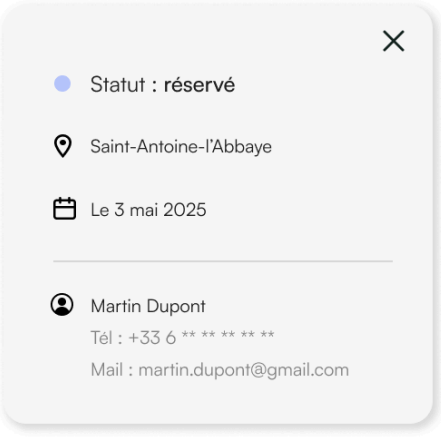
Page de réglementation



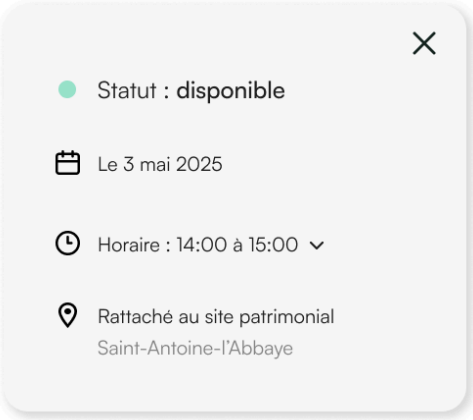
Page profil - Gestion du mot de passe



Dashboard guide amateur/certifié



Pop-up réservation en cours



Pop-up entrer une disponibilité

Sources

- **Persona 1 - Personnes âgées**
  http://www.france-camping-car.fr/statistique-camping-cars.php

  https://fr.statista.com/statistiques/550437/sejours-touristiques-seniors-plus-de-65-ans-francais-en-france/

  https://ffcc.fr/chiffres-cles/

- **Persona 2 - Famille**
  https://www.vie-publique.fr/files/2020-11/9782111450349_VisiterEnFamille_Extrait.pdf

  https://www.radiofrance.fr/franceculture/enfants-au-musee-c-est-la-transmission-du-gout-du-patrimoine-aux-adultes-de-demain-2993260

- **Persona 3 : PMR**
  https://www.statistiques.developpement-durable.gouv.fr/les-pratiques-de-mobilite-des-personnes-en-situation-de-handicap

  mydl.fr

  https://www.chiensguides.fr/actualites/tourisme-face-au-handicap